ai_agent.py

O motor de IA no servidor: rotas, otimização de combustível, cardumes, deriva e integração com o Gemini.

Python · mais de 1.400 linhas · só biblioteca padrão (sem numpy/pandas) · guia dos blocos principais

Visão geral

O **ai_agent.py** é a versão Python do que o *script.js* faz no navegador, mais as partes que usam IA. É chamado pelo *servidor.py* nas rotas */api/route* e */api/chat*. Lê os mesmos arquivos de dados que o navegador (*ocean-data.js*, *world-land.js*), por isso as duas partes dão resultados iguais.

Principais funções do arquivo:

Função	O que faz
`compute_three_routes`	Calcula as 3 rotas (comercial, A*, Copiloto IA) e pede o parecer ao Gemini.
`astar_route / ai_astar_route`	A* por tempo (rota rápida) e A* por combustível (rota Eco).
`optimize_power`	Escolhe a velocidade de cada trecho para gastar o mínimo de combustível dentro do prazo.
`simular_vazamento`	Simula a deriva de uma mancha de óleo.
`prever_cardumes / generate_global_fish_hotspots`	Prevê zonas de cardume (local e global).
`_call_gemini / call_llm_text`	Chamadas REST ao Gemini ou OpenAI, sem SDK.
`llm_chat`	Chat do Copiloto: o modelo decide se chama uma ferramenta (vazamento ou cardumes).

Este guia cobre os blocos principais que pude ler do arquivo; as outras funções (máscara de terra, leitura dos dados, A em Python) seguem a mesma lógica do script.js.*

Blocos de código explicados

Bloco 1 — Otimizador de potência (optimize_power)

ai_agent.py · Python

```
def optimize_power(grid, path, ship_kn, eta_budget_h):
    """Escolhe a velocidade na água (= potência do motor) de cada trecho da rota A* pra
    gastar o mínimo de VLSFO sem estourar o ETA-alvo. Consumo por hora ∝ v³ (lei do
    hélice), então uma hora "comprada" onde a corrente empurra custa bem menos
    combustível do que onde ela freia: o otimizador reduz o motor nos trechos a favor
    e mantém potência nos contrários. Resolvido por relaxação lagrangiana — λ é o
    "preço" de cada hora, ajustado por bisseção até o tempo total caber no ETA."""
    vmin = max(6.0, ship_kn * MIN_SPEED_FRAC)
    speeds = []
    v = ship_kn
    while v >= vmin - 1e-9:
        speeds.append(round(v, 2))
        v -= SPEED_STEP_KN

    segs = []  # por trecho: [(v, horas, toneladas)]
    for i in range(1, len(path)):
        lo1, la1 = path[i - 1]
        lo2, la2 = path[i]
        opts = []
        for v in speeds:
            t = edge_hours(grid, lo1, la1, lo2, la2, v, 1.0)
            if math.isfinite(t):
                opts.append((v, t, fuel_tons_per_hour(v) * t))
        if not opts:  # corrente mais forte que o navio: segue a toda força com avanço mínimo
            t = segment_hours(grid, lo1, la1, lo2, la2, ship_kn)
            opts = [(ship_kn, t, fuel_tons_per_hour(ship_kn) * t)]
        segs.append(opts)

    def solve(lam):
        pick = [min(o, key=lambda x: x[2] + lam * x[1]) for o in segs]
        return pick, sum(p[1] for p in pick)

    pick, total_h = solve(0.0)  # λ=0: tempo não importa -> o mais econômico possível
    if total_h > eta_budget_h:
        lo, hi = 0.0, 1.0
        while solve(hi)[1] > eta_budget_h and hi < 1e6:
            hi *= 2
        for _ in range(45):
            mid = (lo + hi) / 2
            if solve(mid)[1] > eta_budget_h:
                lo = mid
            else:
                hi = mid
        pick, total_h = solve(hi)
```

O que este bloco faz

É a versão Python do mesmo otimizador do *script.js* (*optimizePower*). A ideia:

- **Consumo ∝ v³**: andar um pouco mais devagar economiza muito combustível. Mas o prazo (ETA) limita o quanto dá para reduzir.
- Para cada trecho, calcula tempo e combustível de todas as velocidades possíveis (da de serviço até 60% dela, em passos de 0,25 nó).
- **solve(λ)** escolhe em cada trecho a velocidade que minimiza *combustível* + λ × *tempo*. Com λ = 0 o navio vai o mais devagar possível.
- Se o tempo total passou do ETA, a **bisseção** (45 repetições) procura o menor λ que faz a viagem caber no prazo.
- Resultado: o motor reduz onde a corrente ajuda e mantém força onde ela atrapalha.

Bloco 2 — Zonas de cardume (dados de entrada)

ai_agent.py · Python

```
FISH_ZONES = [
    {"zone": "Costa do Peru / Chile", "system": "Corrente de Humboldt", "lat": (-42, -5), "lon": (-82, -70),
     "species": "Anchoveta-peruana", "t_opt": 17.0, "t_sd": 4.0, "upwelling": True, "prior": 1.0, "biomass": 180000},
    {"zone": "Costa da Namíbia / África do Sul", "system": "Corrente de Benguela", "lat": (-35, -15), "lon": (8, 19),
     "species": "Sardinha-sul-africana", "t_opt": 16.0, "t_sd": 4.0, "upwelling": True, "prior": 0.9, "biomass": 90000},
    # ... mais 6 zonas (Canárias, Califórnia, Grand Banks, Mar do Norte, Golfo de Bengala, Índico Equatorial)
]
FISH_PER_ZONE = 4        # focos por zona
FISH_MIN_SPACING = 3.0   # graus entre focos da mesma zona (evita empilhar no mesmo lugar)
```

O que este bloco faz

Cada zona de pesca é uma caixa de latitude/longitude com: o sistema de corrente, a espécie provável, a **temperatura ótima** (*t_opt*) e a tolerância (*t_sd*) da espécie, se é zona de **ressurgência** (água fria que traz nutrientes) e um peso de produtividade (*prior*: Humboldt é a mais produtiva do planeta).

O *biomass* é só uma escala **ilustrativa** (não há dados reais de estoque). São escolhidos 4 focos por zona, com pelo menos 3° de distância entre eles.

Bloco 3 — Pontuação de cada célula de mar

ai_agent.py · Python

```
        scored = []
        for lon, lat, t0 in cells:
            te, tw = temp(lon + 1, lat), temp(lon - 1, lat)
            tn, ts = temp(lon, lat + 1), temp(lon, lat - 1)
            gx = abs((te if te is not None else t0) - (tw if tw is not None else t0)) / 2
            gy = abs((tn if tn is not None else t0) - (ts if ts is not None else t0)) / 2
            front = min(1.0, math.hypot(gx, gy) / 1.5)  # 1,5 °C/grau já é frente forte
            cold = min(1.0, max(0.0, mean_t - t0) / 4.0) if z["upwelling"] else 0.0
            fit = math.exp(-((t0 - z["t_opt"]) ** 2) / (2 * z["t_sd"] ** 2))
            u, v = cur.sample(lon, lat)
            speed_kn = math.hypot(u, v) * 1.94384
            calm = 1.0 / (1.0 + speed_kn)
            score = (0.4 * front + 0.3 * cold + 0.3 * fit) * (0.6 + 0.4 * calm) * z["prior"]
```

O que este bloco faz

Em cada célula de água da zona, calcula quatro sinais e combina numa nota (*score*):

- **front** (40%): *frente térmica*, a mudança de temperatura entre células vizinhas. Onde duas águas se encontram, o plâncton se concentra.
- **cold** (30%): água mais fria que a média da zona (ressurgência, só nas zonas costeiras).
- **fit** (30%): quanto a temperatura se aproxima da ideal da espécie (curva gaussiana).
- **calm**: corrente lenta favorece a agregação dos peixes; multiplica a nota por 0,6 a 1,0.

No fim a nota é multiplicada pelo peso *prior* da zona.

Bloco 4 — Chamada ao Gemini (REST, com vários modelos)

ai_agent.py · Python

```
def _call_gemini(body_text, gem_key):
    global _gemini_last_ok
    ordem = [m for m in MODELOS_GEMINI if m not in _gemini_dead]
    if _gemini_last_ok in ordem:
        ordem.remove(_gemini_last_ok)
        ordem.insert(0, _gemini_last_ok)

    inicio = time.monotonic()
    for model_name in ordem:
        restante = LLM_TOTAL_BUDGET_S - (time.monotonic() - inicio)
        if restante < 2:
            print("[GEMINI] tempo total esgotado — usando a simulação otimizada.")
            return None
        model_id = model_name.strip().removeprefix("models/")
        url = ("https://generativelanguage.googleapis.com/v1beta/models/"
               f"{model_id}:generateContent")
        # chave no cabeçalho (e não na URL), para não vazar em logs/mensagens de erro
        res = _http_post_json(url, {"contents": [{"parts": [{"text": body_text}]}]},
                              headers={"x-goog-api-key": gem_key},
                              timeout=min(LLM_TIMEOUT_S, restante))
        if "_error" in res:
            status = res.get("_status")
            print(f"[ERRO GEMINI DETALHADO] ({model_name}, status {status}): {res['_error']}")
            if status == 404:
                _gemini_dead.add(model_name)
            if status in _GEMINI_RETRY_STATUS:
                continue
            return None
        try:
            texto = res["candidates"][0]["content"]["parts"][0]["text"]
        except Exception as e:
            print(f"[ERRO GEMINI DETALHADO] ({model_name}): resposta sem texto — {e}")
            continue
        if _gemini_last_ok != model_name:
            print(f"[GEMINI] respondendo com o modelo {model_name}")
        _gemini_last_ok = model_name
        return texto

    print("[GEMINI] nenhum modelo respondeu — usando a simulação otimizada.")
    return None
```

O que este bloco faz

Tenta os modelos da lista *MODELOS_GEMINI* até um responder:

- o último modelo que funcionou vai para o início da fila; modelos que deram 404 (aposentados) são descartados até o servidor reiniciar;
- há um **orçamento total de 45 s** (cada modelo, no máximo 20 s), para a interface nunca ficar esperando demais;
- erros 404, 429 (cota), 5xx (sobrecarga) ou de rede levam ao próximo modelo; erros de chave (400/401/403) encerram na hora, pois falhariam igual em todos;
- a chave vai no **cabeçalho** *x-goog-api-key* (e não na URL), para não aparecer em mensagens de erro;
- se nada funcionar, devolve *None* e o sistema usa o texto calculado localmente.

Bloco 5 — Requisição HTTP simples

ai_agent.py · Python

```
def _http_post_json(url, payload, headers=None, timeout=25):
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url, data=data,
        headers={"Content-Type": "application/json", **(headers or {})},
        method="POST",
    )
    try:
        with urllib.request.urlopen(req, timeout=timeout) as resp:
            return json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        try:
            return {"_error": e.read().decode("utf-8"), "_status": e.code}
        except Exception:
            return {"_error": str(e), "_status": e.code}
    except Exception as e:  # timeout, sem rede, DNS...
        return {"_error": str(e), "_status": None}
```

O que este bloco faz

Envia um JSON por POST usando só a biblioteca padrão (*urllib*), sem instalar nada. Nunca lança exceção: em qualquer falha devolve um dicionário com **_error** e **_status** (o código HTTP, ou *None* para timeout/sem rede), que o chamador interpreta.

Bloco 6 — Escolha entre Gemini e OpenAI

ai_agent.py · Python

```
def call_llm_text(prompt, system=None):
    gem_key = os.environ.get("GEMINI_API_KEY")
    oai_key = os.environ.get("OPENAI_API_KEY")

    if gem_key:
        body_text = (system + "\n\n" + prompt) if system else prompt
        return _call_gemini(body_text, gem_key)

    if oai_key:
        msgs = ([{"role": "system", "content": system}] if system else []) + \
               [{"role": "user", "content": prompt}]
        res = _http_post_json(
            "https://api.openai.com/v1/chat/completions",
            {"model": "gpt-4o-mini", "messages": msgs},
            headers={"Authorization": "Bearer " + oai_key},
            timeout=LLM_TIMEOUT_S,
        )
        try:
            return res["choices"][0]["message"]["content"]
        except Exception:
            return None

    return None
```

O que este bloco faz

Ponto único por onde todo texto de IA passa. Se existir *GEMINI_API_KEY*, usa o Gemini; senão, se existir *OPENAI_API_KEY*, usa o OpenAI (gpt-4o-mini); sem nenhuma, devolve *None* e o sistema funciona no modo offline. As chaves vêm do ambiente (carregadas do *.env* pelo servidor.py).

Bloco 7 — Chat do Copiloto com ferramentas

ai_agent.py · Python

```
def llm_chat(message, context=None, history=None):
    context = context or {}
    ctx_txt = ""
    if context.get("lat") is not None and context.get("lon") is not None:
        ctx_txt = "\nPonto atual do mapa: lat=%.3f, lon=%.3f." % (context["lat"], context["lon"])

    hist_txt = ""
    for h in (history or [])[-6:]:
        who = "Usuario" if h.get("role") == "user" else "Copiloto"
        hist_txt += "\n%s: %s" % (who, h.get("text", ""))

    prompt = "%s\nUsuario: %s%s" % (hist_txt, message, ctx_txt)
    raw = call_llm_text(prompt, system=TOOL_SYSTEM_PROMPT)

    if raw is None:
        return _fallback_chat(message, context)

    try:
        data = json.loads(_extract_json(raw))
    except Exception:
        return {"reply": raw.strip(), "tool": None, "data": None}

    tool = data.get("tool")
    args = data.get("args") or {}
    lat = args.get("lat", context.get("lat"))
    lon = args.get("lon", context.get("lon"))

    if tool == "simular_vazamento" and lat is not None and lon is not None:
        volume = float(args.get("volume_ton", 500))
        horas = int(args.get("horas", 24))
        result = simular_vazamento(lat, lon, volume, horas)
        ...
```

O que este bloco faz

Implementa o **“tool calling”** sem depender do formato de cada fornecedor:

- monta o prompt com as últimas 6 mensagens e o ponto atual do mapa;
- o *TOOL_SYSTEM_PROMPT* pede que o modelo responda **só com JSON**: *{"tool": "simular_vazamento", "args": {...}}* para pedir uma ferramenta, ou *{"tool": null, "reply": "..."}* para conversar;
- o código lê o JSON e **executa a ferramenta localmente** (*simular_vazamento* ou *prever_cardumes*), devolvendo texto e dados para o mapa desenhar;
- sem chave, usa *_fallback_chat*, que decide por palavras-chave.